3 Java 17

This chapter covers

- Text blocks
- Switch Expressions
- Records
- Sealed Types

These represent the major new features that have been added to the Java language and platform since the release of Java 11, up to and including Java 17.

NOTE To understand the changes in the Java release methodology since Java 8, it may be a good idea to review the discussion in chapter 1 or appendix A.

As well as the major, user-visible language upgrades, Java 17 contains many internal improvements (especially performance upgrades). However, this chapter focuses on the major features that we expect will change the way that you, the developer, write Java.

3.1 Text Blocks

Since the very first version, Java 1.0, developers have been complaining about Java's strings. Compared to other programming languages, such as Groovy or Scala or Kotlin, Java's strings have sometimes seemed a little primitive.

Java has historically provided only one type of string—the straightforward, double-quoted string in which certain characters (notably `"` and `\`) must be escaped to be used safely. These have, under a surprisingly wide array of circumstances, led to the need to produce convoluted escaped strings, even for very common programming situations.

The *Text Blocks* project has been through several iterations as a preview feature (we discussed preview features briefly in chapter 1) and is now a standard feature in Java 17. It aims to expand the notion of a string in Java syntax by allowing string literals that extend over multiple lines. In turn, that should avoid the need for most of the escape sequences that, historically, Java programmers have found to be an excessive hindrance.

NOTE Unlike various other programming languages, Java Text Blocks do not currently support *interpolation*, although this feature is under active consideration for inclusion in a future version.

As well as helping to free Java programmers from the bother of dealing with excessive escaping of characters, a specific goal of Text Blocks is to allow readable strings of code that are not Java but that need to be embedded in a Java program. After all, how often do you have to include, for example, SQL or JSON (or even XML) in one of your Java programs?

Before Java 17, this process could be painful, indeed, and, in fact, many teams resorted to using an external templating library with all of its additional complexity. Since the arrival of Text Blocks, this is, in many cases, no longer necessary.

Let's see how they work, by considering an SQL query. In this chapter, we're going to use a few examples from financial trading—specifically foreign exchange currency trading (FX). Perhaps we have our customer orders stored in a SQL database that we will access with a query like this:

```
String query = """
        SELECT "ORDER_ID", "QUANTITY", "CURRENCY_PAIR" FROM "ORDERS"
        WHERE "CLIENT_ID" = ?
        ORDER BY "DATE_TIME", "STATUS" LIMIT 100;
        """;
```

You should notice two things. First, the Text Block is started and terminated with the sequence `"""`, which was not legal Java prior to version 15. Second, the Text Block can be indented with whitespace at the start of each line—and the whitespace will be ignored.

If we print out the `query` variable, then we get exactly the string we constructed, as shown here:

```
SELECT "ORDER_ID", "QUANTITY", "CURRENCY_PAIR" FROM "ORDERS"
WHERE "CLIENT_ID" = ?
ORDER BY "DATE_TIME", "STATUS" LIMIT 100;
```

This happens because a Text Block is a constant expression (of type String), just the same as a string literal. The difference is that a Text Block is processed by `javac` before recording the constant in the class file as follows:

1. Line terminator characters are translated to LF (\uOOOA), that is, the Unix line-ending convention.
2. Extra whitespace surrounding the block is removed, to allow for extra indentation of Java source code, as per our example.
3. Any escape sequences in the block are interpreted.

These steps are carried out in the above order for a reason. Specifically, interpreting escape sequences last means blocks can include literal escape sequences (such as `\n`) without them being modified or deleted by earlier steps.

NOTE At runtime, there is absolutely no difference between a string constant that was obtained from a literal versus a Text Block. The class file does not record in any way the original source of the constant.

For more details about Text Blocks, please see JEP 378 (**https://openjdk.java.net/jeps/378**). Let's move on and meet the new *Switch Expressions* feature.

3.2 Switch Expressions

Since its earliest versions, Java has supported *switch statements*. Java took a lot of inspiration for its syntax from the forms present in C and C++, and the `switch` statement is no exception, as shown next:

```
switch(month) {
  case 1:
    System.out.println("January");
    break;
  case 2:
    System.out.println("February");
    break;
  // ... and so on
}
```

In particular, Java's switch statement inherited the property that if a `case` doesn't end with `break`, execution will continue after the next `case`. This rule allows the grouping of cases that need identical handling, like this:

```
switch(month) {
  case 12:
  case 1:
```

```
  case 2:
    System.out.println("Winter, brrrr");
    break;
  case 3:
  case 4:
  case 5:
    System.out.println("Spring has sprung!");
    break;
  // ... and so on
}
```

Convenience for this situation, though, brought with it a dark and buggy side. Omitting a single `break` is an easy mistake for programmers, both new and old, and often introduced errors. In our example, we would get the wrong answer because excluding our first `break` would have resulted in messages for both winter and spring.

Switch statements are also clunky when trying to capture a value for later use. For example, if we wanted to grab that message for use elsewhere, instead of printing it, we'd have to set up a variable outside the `switch`, set it correctly in each branch, and potentially ensure after the `switch` that we actually set the value; something like this:

```
String message = null;
switch(month) {
  case 12:
  case 1:
  case 2:
    message = "Winter, brrrr";
    break;
  case 3:
  case 4:
  case 5:
    message = "Spring has sprung!";
    break;
  // ... and so on
}
```

Much like a missed `break`, we now must ensure every case properly sets the message variable or risk a bug report in our future. Surely, we can do better.

Switch Expressions, introduced in Java 14 (JEP 361), provide alternatives to address these shortcomings, while also acting to open future language frontiers. This aim includes

helping to close a linguistic gap with more functionally oriented languages (e.g., Haskell, Scala, or Kotlin). A first version of Switch Expressions is more concise, as shown here:

```
String message = switch(month) {
  case 12:
  case 1:
  case 2:
    yield "Winter, brrrr";
  case 3:
  case 4:
  case 5:
    yield "Spring has sprung!";
  // ... and so on
}
```

In this revised form, we no longer set the variable in each branch. Instead, each case uses the new `yield` keyword to hand our desired value back to assign to the `String` variable, and the expression as a whole *yields a value*—from one case branch or another (and each case branch must result in a yield).

With this example in hand, the name of this new feature—*Switch Expressions* versus the existing *Switch Statement*—takes on more meaning. In programming languages, a *statement* is a piece of code executed for its side effect. An *expression* refers instead to code executed to produce a value. `switch` prior to Java 14 was only a side-effecting statement, but now it can produce values when used as an expression.

Switch Expressions also bring another even more concise syntax, which may well prove to be more widely adopted, as shown here:

```
String message = switch(month) {
  case 1, 2, 12  -> "Winter, brrrr";
  case 3, 4, 5   -> "Spring has sprung!";
  case 6, 7, 8   -> "Summer is here!";
  case 9, 10, 11 -> "Fall has descended";
  default        -> {
    throw new IllegalArgumentException("Oops, that's not a month");
  }
}
```

The `->` indicates we're in a switch expression, so those cases don't need an explicit `yield`. Our `default` case shows how a block enclosed in `{}` can be used where we don't

have a single value. If you're using the value of a switch expression (as we are by assigning it to `message`), multiline cases must either `yield` or `throw`.

But the new labeling format isn't just more helpful and shorter—it solves real problems. For one, multiple cases are directly supported by the comma-delimited list after the `case`. This solves the problem that previously required dangerous switch fall-through. A switch expression in the new labeling syntax never falls through, closing off that stumbling block for everyone.

The added safeguards don't end there. Another common way to mess up your switch statements is to miss a case you should have handled. If we remove the `default` line from the previous example, we get a compile error, as shown next:

```
error: the switch expression does not cover all possible input values
   String message = switch(month) {
                    ^
```

Unlike switch statements, Switch Expressions must handle every possible case for your input type, or your code won't even compile. That's an excellent guarantee to help you cover all the bases. It also combines nicely with Java's enums, as we can see if we rewrite the switch to use typesafe constants rather than `int`s as follows:

```
String message = switch(month) {
   case JANUARY, FEBRUARY, DECEMBER  -> "Winter, brrrr";
   case MARCH, APRIL, MAY            -> "Spring has sprung!";
   case JUNE, JULY, AUGUST           -> "Summer is here!";
   case SEPTEMBER, OCTOBER, NOVEMBER -> "Fall has descended";
};
```

This new capability is useful as a standalone feature, because it allows us to simplify a very common case of the use of `switch`, behaving a bit like a function, yielding an output value based on the input value. In fact, the rule for Switch Expressions is that every possible input value must be guaranteed to produce an output value.

NOTE If all the possible enum constants are present in a switch expression, the match is *total* and it is not necessary to include a `default` case—the compiler can use the exhaustiveness of the enum constants.

However, for Switch Expressions that take, for instance, an `int`, we must include a `default` clause as it is not feasible to list all approximately four billion possible values.

Switch Expressions are also a stepping-stone toward a major feature, Pattern Matching, in a possible future version of Java, which we will discuss both later in this chapter and later in the book. For now, let's move on to meet the next new feature, Records.

3.3 Records

Records are a new form of Java class designed to do the following:

- Provide a first-class means for modeling data-only aggregates
- Close a possible gap in Java's type system
- Provide language-level syntax for a common programming pattern
- Reduce class boilerplate

The ordering of these bullet points is important, and in fact, Records are more about language semantics than they are about boilerplate reduction and syntax (although the second aspect is what many developers tend to focus on). Let's start by explaining the basic idea of what a Java record is.

The idea of Records is to extend the Java language and create a way to say that a class is "the fields, just the fields, and nothing but the fields." By making that statement about our class, the compiler can help us by creating all the methods automatically and having all the fields participate in methods like `hashCode()`.

NOTE This is the way that the semantics "a record is a transparent carrier of the fields" defines the syntax: "accessor methods and other boilerplate are automatically derived from the record definition."

To see how it shows up in day-to-day programming, remember that one of the most common complaints about Java is that you need to write a lot of code for a class to be useful. Quite often we need to write

- `toString()`
- `hashCode()` and `equals()`
- Getter methods
- Public constructor

and so on.

For simple domain classes, these methods are usually boring, repetitive, and the kind of thing that could easily be mechanically generated (and IDEs often provide this capability), but until we had Records, the language didn't provide any way to do this directly.

This frustrating gap is actually worse when we're reading someone else's code. For example, it might look like the author is using an IDE-generated `hashCode()` and `equals()` that uses all the fields of the class, but how can we be sure without checking each line of the implementation? What happens if a field is added during refactoring and the methods are not regenerated?

Records solve these problems. If a type is declared as a record, it is making a strong statement, and the compiler and runtime will treat it accordingly. Let's see it in action.

To really explain this feature fully, we need a nontrivial example domain, so let's continue to use FX currency trading. Don't worry if you're not familiar with the concepts used in this area—we'll explain what you need to know as we go along. Later in the book, we're going to continue the theme of financial examples, so this is a good place to get started.

Let's walk through how we can use Records and a few other features to improve our modeling of the domain and get cleaner, less verbose, and simpler code as a result. Consider an order that we want to place when trading FX. The basic order type might consist of the following:

- Number of units I'm buying or selling (in millions of currency units)
- The "side"—whether I'm buying or selling (often called *Bid* and *Ask*)
- The currencies I'm exchanging (the *currency pair*)
- The time I placed my order
- How long my order is good for before it times out (the *time-to-live* or *TTL*)

So, if I have £1M and want to sell it for US dollars within the next second, and I want $1.25 for each £, then I am "buying the GBP/USD rate at $1.25 now, good for 1s." In Java, we might declare a domain class like this (we're calling it "classic" to call out that we have to do this with a class for now—better ways are coming):

```
public final class FXOrderClassic {
    private final int units;
    private final CurrencyPair pair;
    private final Side side;
    private final double price;
    private final LocalDateTime sentAt;
    private final int ttl;

    public FXOrderClassic(int units, CurrencyPair pair, Side side,
                          double price, LocalDateTime sentAt, int ttl) {
        this.units = units;
```

```
    this.pair = pair; // CurrencyPair is a simple enum
    this.side = side; // Side is a simple enum
    this.price = price;
    this.sentAt = sentAt;
    this.ttl = ttl;
}

public int units() {
    return units;
}

public CurrencyPair pair() {
    return pair;
}

public Side side() {
    return side;
}

public double price() {
    return price;
}

public LocalDateTime sentAt() {
    return sentAt;
}

public int ttl() {
    return ttl;
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;

    FXOrderClassic that = (FXOrderClassic) o;

    if (units != that.units) return false;
    if (Double.compare(that.price, price) != 0) return false;
    if (ttl != that.ttl) return false;
    if (pair != that.pair) return false;
    if (side != that.side) return false;
    return sentAt != null ? sentAt.equals(that.sentAt) :
                        that.sentAt == null;
}
```

```
    @Override
    public int hashCode() {
        int result;
        long temp;
        result = units;
        result = 31 * result + (pair != null ? pair.hashCode() : 0);
        result = 31 * result + (side != null ? side.hashCode() : 0);
        temp = Double.doubleToLongBits(price);
        result = 31 * result + (int) (temp ^ (temp >>> 32));
        result = 31 * result + (sentAt != null ? sentAt.hashCode() : 0);
        result = 31 * result + ttl;
        return result;
    }

    @Override
    public String toString() {
        return "FXOrderClassic{" +
                "units=" + units +
                ", pair=" + pair +
                ", side=" + side +
                ", price=" + price +
                ", sentAt=" + sentAt +
                ", ttl=" + ttl +
                '}';
    }
}
```

That's a lot of code, but it means that my order can be created like this:

```
var order = new FXOrderClassic(1, CurrencyPair.GBPUSD, Side.Bid,
                               1.25, LocalDateTime.now(), 1000);
```

But how much of the code to declare the class is really necessary? In older versions of Java, most developers would probably just declare the fields and then use their IDE to autogenerate all the methods. Let's see how Records improve the situation.

NOTE Java doesn't provide any way to talk about a data aggregate other than by defining a class, so it is clear that any type containing "just the fields" will be a class.

The new concept is a *record class* (or usually just record). This is an immutable (in the usual "all fields are final" Java sense), transparent carrier for a fixed set of values, known as the *record components*. Each component gives rise to a final field that holds the pro-

vided value and an accessor method to retrieve the value. The field name and the accessor name match the name of the component.

The list of fields provides a *state description* for the record. In a general class, there might be no relation between a field `x`, the constructor argument `x`, and the accessor `x()`, but in a record, they are *by definition* talking about the same thing—a record *is* its state.

To allow us to create new instances of record classes, a constructor is also generated—called the *canonical constructor*—which has a parameter list that exactly matches the declared state description. The Java language also now provides concise syntax for declaring Records, in which all the programmer needs to do is to declare the component names and types that make up the record, like this:

```
public record FXOrder(int units,
                      CurrencyPair pair,
                      Side side,
                      double price,
                      LocalDateTime sentAt,
                      int ttl) {}
```

By writing this record declaration, we are not just saving some typing, we are making a much stronger, semantic statement. The `FXOrder` type *is* just the state provided, and any instance *is* just a transparent aggregate of the field values.

If we now examine the class file with `javap` (which we will meet properly in chap-ter 4), we can see that the compiler has autogenerated a bunch of boilerplate code for us:

```
$ javap FXOrder.class
Compiled from "FXOrder.java"
public final class FXOrder extends java.lang.Record {
  public FXOrder(int, CurrencyPair, Side,
                 double, java.time.LocalDateTime, int);

  public java.lang.String toString();
  public final int hashCode();
  public final boolean equals(java.lang.Object);
  public int units();
  public CurrencyPair pair();
  public Side side();
  public double price();
  public java.time.LocalDateTime sentAt();
```

```
  public int ttl();
}
```

This looks remarkably like the set of methods we had to write in the code for the class-based implementation. In fact, the constructor and accessor methods all behave exactly as before. However, methods like `toString()` and `equals()` use an implementation that might be surprising to some developers, as shown here:

```
public java.lang.String toString();
    Code:
      0: aload_0
      1: invokedynamic #51,  0          // InvokeDynamic #0:toString:
                                        // (LFXOrder;)Ljava/lang/String;
      6: areturn
```

That is, the `toString()` method (and `equals()` and `hashCode()`) are implemented using an `invokedynamic`-based mechanism. This is a powerful technique that we will meet later in the book (in chapters 4 and 16).

We can also see that there is a new class, `java.lang.Record`, that will act as the super-type for all record classes. It is abstract and declares `equals()`, `hashCode()` and `toString()` to be abstract methods. The `java.lang.Record` class cannot be directly extended, as we can see by trying to compile some code like this:

```
public final class FXOrderClassic extends Record {
    private final int units;
    private final CurrencyPair pair;
    private final Side side;
    private final double price;
    private final LocalDateTime sentAt;
    private final int ttl;

    // ... rest of class elided
}
```

The compiler will reject this attempt:

```
$ javac FXOrderClassic.java
FXOrderClassic.java:3: error: records cannot directly extend Record
public final class FXOrderClassic extends Record {
```

```
        ^
1 error
```

The only way to get a record is to explicitly declare one and have `javac` create the class file. This also ensures that all record classes are created as final.

As well as the autogeneration of methods and boilerplate reduction, a couple of other core Java features also have special characteristics when applied to Records. First, Records must obey a special contract regarding the `equals()` method: if a record `R` has components `c1`, `c2`, ... `cn`, and if a record instance is copied as follows:

```
R copy = new R(r.c1(), r.c2(), ..., r.cn());
```

then it must be the case that `r.equals(copy)` is `true`. Note that this invariant is in addition to the usual familiar contract regarding `equals()` and `hashCode()`—it does not replace it.

At this point, let's move on to talk about some of the more design-level aspects of the Records feature. To do so, it's helpful to recall how enums work in Java. An enum in Java is a special form of class that implements a design pattern (*finitely many typesafe instances*) but with minimal syntax overhead—the compiler generates a bunch of code for us.

Similarly, a record in Java is a special form of class that implements a pattern (*Data Carrier* aka *Just Holds Fields*) with minimal syntax. All of the boilerplate code that we expect will be autogenerated for us by the compiler. However, although the simple concept of a Data Carrier class that just holds fields makes intuitive sense, what does that really mean in detail?

When Records were first being discussed, a lot of possible different designs were considered. For example:

- Boilerplate reduction of POJOs
- Java Beans 2.0
- Named tuples
- Product types (a form of *algebraic data type*)

These possibilities were discussed by Brian Goetz in his original design sketch (**http://mng.bz/M5j8**) in some detail. Each design option comes with additional sec-

ondary questions that follow from the choice of the design center for Records, questions such as:

- Can Hibernate proxy them?
- Are they fully compatible with classic Java Beans?
- Do they support name erasure/"shape malleability"?
- Will they come with Pattern Matching and destructuring?

It would have been plausible to base the Records feature on any one of the above four approaches—each has advantages and disadvantages. However, the final design decision is that Records are *named tuples*. This is partially driven by a key design idea in Java's type system— *nominal typing*. Let's take a closer look at this key idea.

3.1 Nominal typing

The nominal approach to static typing is the idea that every piece of Java storage (variables, fields) has a definite type and that each type has a name, which should be (at least somewhat) meaningful to humans.

Even in the case of anonymous classes, the types still have names—it's just that the compiler assigns the names and they are not valid names for types in the Java language (but are still OK within the JVM). For example, we can see this in jshell:

```
jshell> var o = new Object() {
   ...>   public void bar() { System.out.println("bar!"); }
   ...> }
o ==> $0@37f8bb67

jshell> var o2 = new Object() {
   ...>   public void bar() { System.out.println("bar!"); }
   ...> }
o2 ==> $1@31cefde0

jshell> o = o2;
|  Error:
|  incompatible types: $1 cannot be converted to $0
|  o = o2;
|      ^^
```

Notice that even though the anonymous classes were declared in exactly the same way, the compiler still produced two different anonymous classes, `$0` and `$1`, and would

not allow the assignment, because in the Java type system, the variables have different types.

NOTE There are other (non-Java) languages where the overall shape of class (e.g., what fields and methods it has) can be used as the type (rather than an explicit type name). This is called *structural typing*.

It would have been a major change if Records had broken with Java's heritage and brought in structural typing for Records. As a result, the "Records are nominal tuples" design choice means that we expect that Records will work best where we might use tuples in other languages. This includes use cases such as compound map keys, or to simulate multireturn from a method. An example compound map key might look like this:

```
record OrderPartition(CurrencyPair pair, Side side) {}
```

Conversely, Records will not necessarily work well as a replacement for existing code that currently uses Java Beans. A number of reasons exist, notably that Java Beans are mutable whereas Records are not and that they have different conventions for their accessors. Records name their accessor methods the same as the field names (possible because field and method names are separately namespaced in Java) whereas Beans prepend `get` and `set`.

Records do allow some additional flexibility above and beyond the simple, single-line declaration form because they are genuine classes. Specifically, the developer can define additional methods, constructors, and static fields apart from the autogenerated defaults. However, these capabilities should be used carefully. Remember that the design intent of Records is to allow the developer to group related fields as a single, immutable data item.

One example of a use additional method that a record might create is a static factory method to simulate default values for some of the record parameters. Another example might be a `Person` class (with immutable date of birth) that might define a `currentAge()` method.

A good rule of thumb is: the more tempting it is to add a lot of additional methods and so on to the basic Data Carrier (or to make it implement several interfaces), the more likely it is that a full class should be used, rather than a record.

3.2 Compact record constructors

One important possible exception to the simplicity/"full class" rule of thumb is the use of *compact constructors*, which are described like this in the language specification:

The formal parameters of a compact constructor of a record class are implicitly declared. They are given by the derived formal parameter list of the record class.

The intention of a compact constructor declaration is that only validation and/or normalization code need be given in the body of the canonical constructor; the remaining initialization code is supplied by the compiler.

—Java Language Specification

For example, we might want to validate orders to make sure that they don't attempt to buy or sell negative quantities or set an invalid time-to-live as follows:

```
public record FXOrder(int units, CurrencyPair pair, Side side,
                      double price, LocalDateTime sentAt, int ttl) {
   public FXOrder {
      if (units < 1) {
         throw new IllegalArgumentException(
               "FXOrder units must be positive");
      }
      if (ttl < 0) {
         throw new IllegalArgumentException(
               "FXOrder TTL must be positive, or 0 for market orders");
      }
      if (price <= 0.0) {
         throw new IllegalArgumentException(
               "FXOrder price must be positive");
      }
   }
}
```

One advantage that Java Records have over the anonymous tuples found in other languages is that the constructor body of a record allows for code to be run when Records are created. This allows for validation to occur (and exceptions to be thrown if an invalid state is passed). This would not be possible in purely structural tuples.

It might also make sense to use static factory methods within the body of the record, for example, to work around the lack of default parameter values in Java. In our trading ex-

ample we might include a static factory like this

```
public static FXOrder of(CurrencyPair pair, Side side, double price) {
    var now = LocalDateTime.now();
    return new FXOrder(1, pair, side, price, now, 1000);
}
```

to declare a quick way to create orders with defaulted parameters. This could also be declared as an alternate constructor, of course. The developer should choose which approach makes sense to them in each circumstance.

One other use for alternate constructors is to create Records for use as compound map keys, as in this example:

```
record OrderPartition(CurrencyPair pair, Side side) {
    public OrderPartition(FXOrder order) {
        this(order.pair(), order.side());
    }
}
```

The type `OrderPartition` can then be easily used as a map key. For instance, we might want to construct an order book for use in our trade matching engine, like so:

```
public final class MatchingEngine {
    private final Map<OrderPartition, RankedOrderBook> orderBooks =
                                          new TreeMap<>();

    public void addOrder(final FXOrder o) {
        orderBooks.get(new OrderPartition(o)).addAndRank(o);
        checkForCrosses(o.pair());
    }

    public void checkForCrosses(final CurrencyPair pair) {
        // Do any buy orders match with sells now?
    }

    // ...
}
```

Now, when a new order is received, the `addOrder()` method extracts the appropriate order partition (consisting of a tuple of the currency pair and buy/sell side) and uses it to

add the new order to the appropriate price-ranked order book. The new order might match against existing orders already on the books (which is called "crossing" of orders), so we need to check if it does in the `checkForCrosses()` method.

Sometimes we might want not to use the compact constructor and instead have a full, explicit canonical constructor. This signals that we need to do actual work in the constructor—and the number of use cases for this with simple Data Carrier classes is small. However, for some situations, like the need to make defensive copies of incoming parameters, this is necessary. As a result, the possibility of an explicit canonical constructor is permitted by the compiler—but think very carefully before making use of this approach.

Records are intended to be simple Data Carriers, a version of tuples that fits into Java's established type system in a logical and consistent manner. This will help many applications make domain classes clearer and smaller. It will also help teams eliminate many hand-coded implementations of the underlying pattern. It should also reduce or remove the need for libraries like Lombok.

Many developers are already reporting significant improvements when starting to use Records. They also combine extremely well with another new feature that also arrived in Java 17—Sealed Types.

3.4 Sealed Types

Java's enums are a well-known language feature. They allow the programmer to model a finite set of alternatives that represent all possible values of a type—effectively typesafe constants.

To continue our FX example, let's consider an `OrderType` enum to denote different types of order:

```
enum OrderType {
    MARKET,
    LIMIT
}
```

This represents two possible types of FX order: a *market* order that will take whatever the current best price is, and a *limit* order that will execute only when a specific price is available. The platform implements enums by having the Java compiler automatically generate a special form of class type.

NOTE The runtime actually treats the library type `java.lang.Enum` (which all enum classes directly extend) in a slightly special way compared to other classes, but the details of this need not concern us here.

Let's decompile this enum and see what the compiler generates as follows:

```
$ javap -c -p OrderType.class
final class OrderType extends java.lang.Enum<OrderType> {
  public static final OrderType MARKET;

  public static final OrderType LIMIT;

  ...
  // Private constructor
}
```

Within the class file, all the possible values of the enum are defined as `public static final` variables, and the constructor is private, so additional instances cannot be constructed.

In effect, an enum is like a generalization of the Singleton pattern, except that instead of being only one instance of the class, there are a finite number. This pattern is extremely useful, especially because it gives us a notion of *exhaustiveness*—given a not-null `OrderType` object, we can know for sure that it is either the `MARKET` or the `LIMIT` instance.

However, suppose we want to model many different orders in Java 11. We must choose between two unpalatable alternatives. First, we can choose to have a single implementing class (or record), `FXOrder`, with a state field holding the actual type. This pattern works because the state field is of enum type and provides the bits that indicate which type is really meant for this specific object. This is obviously suboptimal, because it requires the application programmer to keep track of bits that are really the proper concern of the type system. Alternatively, we can declare an abstract base class, `BaseOrder`, and have concrete types, `MarketOrder` and `LimitOrder`, that subclass it.

The issue here is that Java has always been designed as an open language that is extensible by default. Classes are compiled at one time, and subclasses can be compiled years (or even decades) later. As of Java 11, the only class inheritance constructs permitted in the Java language are open inheritance (default) and no inheritance (`final`).

Classes can declare a package-private constructor, which effectively means "can only be extended by package-mates," but nothing in the runtime prevents users from creating new classes in packages that are not part of the platform, so this is an incomplete protection at best.

If we define a `BaseOrder` class, then nothing prevents a third party from creating a `EvilOrder` class that inherits from `BaseOrder`. Worse still, this unwanted extension can happen years (or decades) after the `BaseOrder` type was compiled, which is hugely undesirable.

The conclusion is that until now, developers have been constrained and must use a field to hold the actual type of the `BaseOrder` if they want to be future-proof. Java 17 has changed this state of affairs, by allowing a new way to control inheritance in a more fine-grained way: the *sealed type*.

NOTE This capability is present in several other programming languages in various forms and has become somewhat fashionable in recent years, although it is actually quite an old idea.

In its Java incarnation, the concept that sealing expresses is the idea that a type can be extended, but only by a known list of subtypes and no others. Let's look at the new syntax for a simple example of a `Pet` class (we'll return to FX examples in a moment):

```
public abstract sealed class Pet {
    private final String name;

    protected Pet(String name) {
        this.name = name;
    }

    public String name() {
        return name;
    }

    public static final class Cat extends Pet {
        public Cat(String name) {
            super(name);
        }

        void meow() {
            System.out.println(name() +" meows");
        }
    }
```

```
    public static final class Dog extends Pet {
      public Dog(String name) {
        super(name);
      }

      void bark() {
        System.out.println(name() +" barks");
      }
    }
}
```

The class `Pet` is declared as `sealed`, which is not a keyword that has been permitted in Java until now. Unqualified, `sealed` means that the class can be extended only inside the current compilation unit. Therefore, the subclasses have to be nested within the current class. We also declare `Pet` to be `abstract` because we don't want any general `Pet` instances, only `Pet.Cat` and `Pet.Dog` objects. This provides us with a nice way to implement the object-oriented (OO) modeling pattern we described earlier, without the drawbacks that we discussed.

Sealing can also be used with interfaces, and it's quite possible that the interface form will be more widely used in practice than the class form. Let's take a look at what happens when we want to use sealing to help model different types of FX orders:

```
public sealed interface FXOrder permits MarketOrder, LimitOrder {
    int units();
    CurrencyPair pair();
    Side side();
    LocalDateTime sentAt();
}

public record MarketOrder(int units,
                          CurrencyPair pair,
                          Side side,
                          LocalDateTime sentAt,
                          boolean allOrNothing) implements FXOrder {

    // constructors and factories elided
}

public record LimitOrder(int units,
                         CurrencyPair pair,
                         Side side,
```

```
                LocalDateTime sentAt,
                double price,
                int ttl) implements FXOrder {

    // constructors and factories elided
}
```

There are several things to notice here. First, `FXOrder` is now a `sealed` interface. Second, we can see the use of a second new keyword, `permits`, which allows the developer to list the permissible implementations of this sealed interface—and our implementations are Records.

NOTE When you use `permits`, the implementing classes do not have to live within the same file and can be separate compilation units.

Finally, we have the nice bonus—because `MarketOrder` and `LimitOrder` are proper classes, they can have behaviors specific to their types. For example, a market order just takes the best price available immediately and does not need to specify a price. On the other hand, a limit order needs to specify the price that the order will accept and how long it is prepared to wait to try to achieve it (the time-to-live or *TTL*). This would not have been straightforward if we were using a field to indicate the "real type" of the object, because all methods for all subtypes would have to be present on the base type or force us to use ugly downcasts.

If we now program with these types, we know that any `FXOrder` instance that we encounter must either be a `MarketOrder` or a `LimitOrder`. What's more, the compiler can use this information, too. Library code can now safely assume that these are the only possibilities, and this assumption cannot be violated by client code.

Java's OO model represents the two most fundamental concepts of the relationship between types. Specifically, `"Type `X IS-A Y`"` and `"Type `X HAS-A Y`."` Sealed Types represent an object-oriented concept that previously could not be modeled in Java: `"Type `X IS-EITHER-A Y OR Z`."` Alternatively, they can also be thought of as

- A halfway house between `final` and open classes
- The enum pattern applied to types instead of instances

In terms of OO programming theory, they represent a new kind of formal relationship, because the set of possible types for `o` is the *union* of `Y` and `Z`. Accordingly, this is

known as a *union type* or *sum type* in various languages, but don't be confused—they are different from C's `union`.

For example, Scala programmers can implement a similar idea using case classes and their own version of the `sealed` keyword (and we'll meet Kotlin's take on this idea later).

Beyond the JVM, the Rust language also provides a notion of *disjoint union* types, although it refers to them using the `enum` keyword, which is potentially *extremely* confusing for Java programmers. In the functional programming world, some languages (e.g., Haskell) provide a feature called *algebraic data types* that contain sum types as a special case. In fact, the combination of Sealed Types and Records also provides Java 17 with a version of this feature.

On the face of it, these types seem like a completely new concept in Java, but their deep similarity to enums should provide a good starting point for many Java programmers. In fact, something similar to these types already exists in one place: the type of the exception parameter in a multicatch clause.

From the Java Language Specification (JLS 11, section 14.20):

> **The declared type of an exception parameter that denotes its type as a union with alternatives D1 | D2 | ... | Dn is lub(D1, D2, ..., Dn).**

However, in the multicatch case, the true union type cannot be written as the type of a local variable—it is *nondenotable*. We cannot create a local variable typed as the true union type in the case of multicatch.

We should make one final point about Java's Sealed Types: they must have a base class that all the permitted types extend (or a common interface that all permitted types must implement). It is not possible to express a type that is "ISA-String-OR-Integer," because the types `String` and `Integer` have no common inheritance relationship apart from `Object`.

NOTE Some other languages do permit the construction of general union types, but it's not possible in Java.

Let's move on to discuss another new language feature that was delivered in Java 17—a new form of the `instanceof` keyword.

3.5 New form of instanceof

Despite being part of the language since Java 1.0, the `instanceof` operator sometimes gets a certain amount of bad press from some Java developers. In its simplest form, it provides a simple test: `x instanceof Y` returns `true` if the value `x` can be assigned to a variable of type `Y` and `false` otherwise (with the caveat that `null instanceof Y` is `false` for every `Y`).

This definition has been derided as undermining object-oriented design, because it implies a lack of preciseness in the types of objects and possibly in the choice of parameter types. However, in practice, in some scenarios the developer must confront an object that has a type that is not fully known at compile time. For example, consider an object that has been obtained reflectively about which little or nothing is known.

In these circumstances, the appropriate thing to do is to use `instanceof` to check that the type is as expected and then perform a downcast. The `instanceof` test provides a guard condition that ensures that the cast will not cause a `ClassCastException` at runtime. The resulting code looks like this example:

```
Object o = // ...
if (o instanceof String) {
   String s = (String)o;
   System.out.println(s.length());
} else {
   System.out.println("Not a String");
}
```

From the point of view of the developer, the new `instanceof` capability available in Java 17 is very simple—it simply provides a way to avoid the cast, as shown here:

```
if (o instanceof String s) {
   System.out.println(s.length());          (1)
} else {
   System.out.println("Not a String");      (2)
}

// ... More code                             (3)
```

(1) s is in scope on this branch.

❷ s is not in scope on the “else” branch.

❸ s is not in scope once the if statement has ended.

However, although it might not seem that important, we get an important clue from the way that the JEP for this feature was named. JEP 394 was titled “Pattern Matching for instanceof,” and it introduces a new concept—the *pattern*.

NOTE It is very important to understand that this is a different usage of *pattern matching* than that used in text processing and regular expressions.

In this context, a pattern is a combination of the following two things:

1. A predicate (aka test) that will be applied to a value
2. A set of local variables, known as *pattern variables*, that are to be extracted from the value

The key point is that the pattern variables are extracted only if the predicate is successfully applied to the value.

In Java 17, the `instanceof` operator has been extended to take either a type or a *type pattern*, where a type pattern consists of a predicate that specifies a type, along with a single pattern variable.

NOTE We will meet type patterns in more detail in the next section.

As it stands, the upgraded `instanceof` does not seem to be very significant, but it is the first time that patterns have been seen in the Java language, and as we will see, more usages are coming! This is but the first step.

Having completed our tour of new Java 17 language features, it's time to look to the future and return to the subject of preview features.

3.6 Pattern Matching and preview features

In chapter 1, we introduced the concept of preview features, but we couldn’t give a good example of one, because Java 11 didn’t have any preview features! Now that we’re talking about Java 17, we can carry on with the discussion.

In fact, all of the new language features that we’ve met in this chapter, including Switch Expressions, Records, and Sealed Types, went through the same lifecycle. They started

out as preview features and went through one or more rounds of public preview before being delivered as final features. For example, sealed classes were previewed in Java 15, and again in 16, before being delivered as a final feature in Java 17 LTS.

In this section, we're going to meet a preview feature that extends Pattern Matching from `instanceof` to `switch`. Java 17 includes a version of this feature, but only as a first preview version (see chapter 1 for more details on preview features). The syntax is liable to change before the final release (and the feature may even be withdrawn, although this is most unlikely for Pattern Matching).

Let's see how Pattern Matching can be used in a simple case to improve some code that has to deal with objects of unknown type. We can use the new form of `instanceof` to write some safe code like this:

```
Object o = // ...

if (o instanceof String s) {
   System.out.println("String of length:"+ s.length());
} else if (o instanceof Integer i) {
   System.out.println("Integer:"+ i);
} else {
   System.out.println("Not a String or Integer");
}
```

This is quickly going to get cumbersome and verbose, though. Instead, we could introduce type patterns into a `switch` expression, as well as the simple `instanceof` Boolean expressions we already have. In the syntax of the current (Java 17) preview feature, we can rewrite the previous code into a simple form:

```
var msg = switch (o) {
   case String s      -> "String of length:"+ s.length();
   case Integer i     -> "Integer:"+ i;
   case null, default -> "Not a String or Integer";          (1)
};
System.out.println(msg);
```

(1) Null is now allowed as a case label to prevent the possibility of NullPointerException.

For those developers who want to experiment with code like this, we should explain how to build and run with preview features. If we try to compile code like the previous exam-

ple that uses a preview feature, we get an error, as shown next:

```
$ javac ch3/Java17Examples.java
ch3/Java17Examples.java:68: error: patterns in switch statements are a
  preview feature and are disabled by default.

          case String s -> "String of length:"+ s.length();
               ^
  (use --enable-preview to enable patterns in switch statements)
1 error
```

The compiler helpfully hints that we might need to enable preview features, so we try again with the flag enabled:

```
$ javac --enable-preview -source 17 ch3/Java17Examples.java
Note: ch3/Java17Examples.java uses preview features of Java SE 17.
Note: Recompile with -Xlint:preview for details.
```

The story is similar at runtime as well:

```
$ java ch3.Java17Examples
Error: LinkageError occurred while loading main class ch3.Java17Examples
    java.lang.UnsupportedClassVersionError: Preview features are not enabled
  for ch16/Java17Examples (class file version 61.65535). Try running with
  '--enable-preview'
```

Finally, if we include the preview flag, then the code will finally run:

```
$ java --enable-preview ch13.Java17Examples
```

The need to constantly enable the preview features is a pain, but it is designed to protect developers from having any code that uses unfinished features from escaping into production and causing problems there. Similarly, it's important to note the message about class file version that appeared when we tried to run a class containing preview features without the runtime flag. If we have explicitly compiled with preview features, we do not get a standard class file, and most teams should not be running that code in production.

The preview version of Pattern Matching in Java 17 also has functionality to integrate closely with Sealed Types. Specifically, patterns can take advantage of the fact that Sealed Types offer exclusivity of the possible types that can be seen. For example, when processing FX order responses, we may have the following base type:

```
public sealed interface FXOrderResponse
        permits FXAccepted, FXFill, FXReject, FXCancelled {
    LocalDateTime timestamp();
    long orderId();
}
```

We can combine this with a switch expression and type patterns, to give some code like this:

```
FXOrderResponse resp = // ... response from market
var msg = switch (resp) {
    case FXAccepted a  -> a.orderId() + " Accepted";
    case FXFill f      -> f.orderId() + " Filled "+ f.units();
    case FXReject r    -> r.orderId() + " Rejected: "+ r.reason();
    case FXCancelled c -> c.orderId() + " Cancelled";
    case null          -> "Order is null";
};
System.out.println(msg);
```

Note that a) we explicitly include a `case null` to ensure this code is null-safe (and won't throw a `NullPointerException`), and b) we do not need a default. The second point is because the compiler can examine all of the permitted subtypes of `FXOrderResponse` and can conclude that the pattern match is *total*, it covers every possibility that could ever occur and so a default case would be dead code under all circumstances. In the case where the match is not total, and some cases are not covered, a default would be needed.

The first preview also includes *guarded patterns*, which allow a pattern to be decorated with a Boolean guard condition, so the overall pattern matches only if both the pattern predicate and the guard are true. For example, let's suppose we want to see the details only of large filled orders. We can change the fill case in the previous example to some code like this:

```
case FXFill f && f.units() < 100 -> f.orderId() + " Small Fill";
case FXFill f                    -> f.orderId() + " Fill "+ f.units();
```

Note that the more specific case (small orders of less than 100 units) is tested first, and only if it fails does the match attempt the next case, which is the unguarded match for fills. The pattern variable is also already in scope for any guard conditions. We will return to Pattern Matching in chapter 18 when we discuss the future of Java and talk about some features that didn't make it in time for Java 17.

Summary

- Java 17 introduced a number of new features that developers will immediately be able to take advantage of in their own code:
  - Text Blocks for multiline strings.
  - Switch Expressions for a more modern switch experience.
  - Records as transparent carriers of data.
  - Sealed Types—an important new OO modeling concept.
  - Pattern Matching—although not fully delivered as of Java 17, it clearly shows the direction of travel of the language in the coming versions.

Previous chapter
< 2 Java modules

Next chapter
Part 2. Under the hood >